RegulaTrace: Evidence-Anchored Regulatory Obligation Analysis with a Transparent Baseline and a Held-Out Evaluation Protocol

Lakshit Sachdeva Anannya Samudra
Trayee Nayampalli Riddhi Ranjan
NMIMS MPSTME, Mumbai, India

26 October 2026
Working preprint draft

Abstract

Regulatory provisions combine duties, actors, deadlines, conditions, exceptions, and cross-references. An extraction system that returns a plausible paraphrase without a recoverable source location is difficult to evaluate or review. We present RegulaTrace, an implemented, deterministic baseline for evidence-anchored regulatory obligation analysis. It segments supplied text, labels five operational modalities, extracts heuristic fields, retains Unicode code-point offsets into the supplied text, and exposes policy-comparison and version-change triage with persistent human corrections. We also specify a document-level annotation format and evaluator with source-group split checks, one-to-one span matching, candidate detection, modality accuracy, critical-miss rate, and provenance checks. On a transparent 16-case synthetic challenge suite, the current implementation passes all 16 cases, including nine labeled critical cases. This is a software smoke test, not evidence of performance on unseen regulations. We make the implementation and evaluation contract inspectable and identify the annotated corpus, model comparisons, and blinded reviewer study needed before substantive effectiveness claims are warranted.

Keywords: legal NLP; regulatory obligation extraction; deontic modality; provenance; human review; evaluation protocol

1 Introduction

Regulatory text often expresses an action together with who must perform it, when, under what conditions, and with which exceptions. A system that loses one of these constraints can produce an apparently useful obligation record that changes the rule's practical meaning. We therefore treat the source passage and its location as part of the prediction, rather than as an optional explanation.

Prior legal NLP benchmarks have shown the value of task-specific evaluation and document evidence. LexGLUE gathers several legal language-understanding tasks [1]; ContractNLI pairs document-level decisions with supporting contract spans [2]. Regulatory extraction has its own structure: Piskorski and Skotarczak study information-provision requirements in EU legislation, including participants and temporal attributes [4], while Minkova et al. examine deontic modality under limited labels [3]. These works motivate a narrow, inspectable baseline and a held-out study, rather than a broad claim of regulatory understanding.

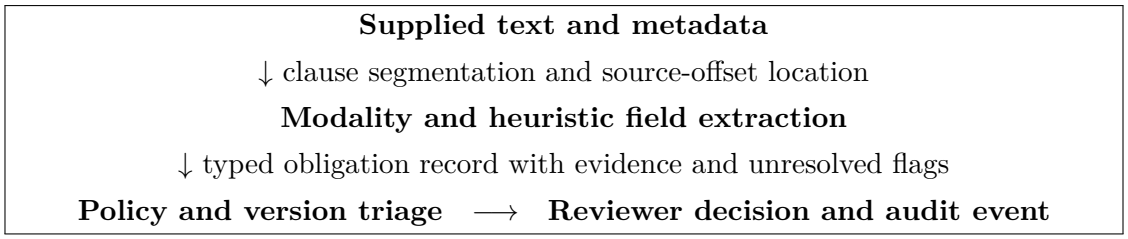


Figure 1: Implemented local pipeline. The policy and version branches provide review signals; they do not issue legal compliance determinations.

This paper makes three bounded contributions. First, it describes a running baseline whose output records retain source offsets, raw evidence, extracted fields, and unresolved flags. Second, it documents review, policy-comparison, and version-triage components as implemented, with their heuristic boundaries explicit. Third, it provides an executable evaluation protocol that separates a small synthetic development suite from a future document-level test set. We do not claim superiority over learned models or analyst workflows: those comparisons have not been run.

2 Task and Evidence Contract

Let a document be a Unicode string T with a source identifier and version metadata. The current system segments T into candidate clauses. For each clause, it proposes a record

$$r = (m, a, x, d, c, e, q, [b, u), v),$$

where m is modality, a actor, x action, d raw deadline expression, c condition, e exception, q quoted evidence, $[b, u)$ a source interval, and v source version. Offsets count Unicode code points; u is exclusive. The operational labels are obligation, prohibition, recommendation, permission, and information. The information label routes descriptive or epistemic statements away from the normative register.

A located record must be checkable against the supplied or extracted text: the interval $T[b : u]$ should recover the retained clause after whitespace normalization. The identifier and evidence quote do not prove that an uploaded document is an authoritative legal source. PDF offsets refer to extracted text, not page coordinates. Cross-references are recorded but are not automatically resolved; fields that require surrounding provisions can remain unresolved.

3 Implemented Baseline

3.1 Ingestion and obligation extraction

The FastAPI application accepts text-based PDF, DOCX, TXT, and Markdown. Scanned PDFs require OCR outside the present pipeline. Clause segmentation uses punctuation, line boundaries, and list markers. Ordered patterns identify modal cues such as *must*, *shall not*, *should*, and *may*; a special rule treats phrases such as *may not be complete* as epistemic information rather than prohibition. Actor and action are derived from text around the cue. Further patterns identify raw deadlines, conditions, exceptions, and cross-references. The action parser strips certain leading timing qualifiers before selecting the operative phrase; this addresses a failure exposed by the bundled breach-notification example.

The source locator searches clauses in document order and returns exact start and end offsets when a normalized clause can be found. It returns a missing value and an unresolved flag when it

cannot locate one. The implementation maintains corresponding Python and browser JavaScript engines. Its numerical confidence is a hand-built heuristic based on cues and completeness, not a calibrated probability.

3.2 Policy comparison and version triage

The current policy comparator reads the first extracted requirement and policy records, then scores actor, action, polarity, timing, exception, and evidence dimensions. Lexical overlap and approximate numeric time conversion feed a weighted score, with polarity and timing failures able to override an otherwise high score. The output is *likely aligned*, *review required*, or *potential mismatch*. It is not a legal conclusion, and it does not implement corpus retrieval or semantic entailment.

Version triage greedily aligns old and new clauses by token overlap. It reports added, removed, and modified clauses, followed by a severity signal and a text-based change explanation. A proposed reason is labeled as an inference; legislative intent cannot be established from two text versions alone. Reordered clauses, inherited context, paraphrases, and definition changes remain difficult for this alignment method.

3.3 Human review and local audit

The application stores analyses and review events in local SQLite. A reviewer can confirm, reject, or correct selected fields. A correction updates the saved record and records before/after values, reviewer, note, and timestamp while retaining the original evidence clause. The UI can reopen a saved case and show its current review state. This is a single-user local workflow without identity management or role-based access.

4 Evaluation Design

4.1 Synthetic development checks

The repository contains a transparent challenge suite: 12 extraction cases, three policy-comparison cases, and one version-change case. Extraction checks include modality, actor, deadline when present, and evidence location. Nine cases are designated critical for the safety diagnostic. We define a *critical miss* as a materially misread or missed duty in those extraction cases, a missed serious policy mismatch, or a missed material version change. This denominator differs from the overall pass rate. The examples were authored with the system and are not an independent sample from legal practice.

4.2 Held-out document protocol

A separate JSON Lines format specifies one document per row, including document ID, source-group ID, split, text, and zero or more annotated normative spans with modality and a pre-assigned critical flag. A descriptive-only document has an empty annotation list. The loader rejects malformed offsets, overlapping gold spans, repeated document IDs, and source or exact-text leakage across train, development, and test splits. Near duplicates still require manual review. Two annotators should independently label test documents and adjudicate disagreements without seeing predictions.

The evaluator matches predicted and gold spans one to one by descending character intersection-over-union, with a prespecified threshold of 0.5. Unmatched gold spans are false negatives and

unmatched predictions are false positives. It reports precision, recall, and F1 for candidate detection; modality accuracy on matched spans; and

$$\mathrm{CMR} = \frac{\#\{\text{critical gold duties unmatched or wrong modality}\}}{\#\{\text{critical gold duties}\}}.$$

When there are no critical gold duties, CMR is undefined and is reported as null. The evaluator also checks whether each predicted evidence interval maps back to the supplied text and emits error rows for review. The report includes the engine version, split, raw counts, and a hash of the selected documents.

Table 1: Current executable checks. These are development and engineering checks, not held-out estimates.

Check	Cases	Passed	Interpretation
Synthetic extraction	12	12	Cue, actor, timing, and provenance checks
Synthetic policy comparison	3	3	Expected triage outcome
Synthetic version change	1	1	Added and modified clauses detected
Critical subset	9	9	Zero misses on selected synthetic cases

5 Observed Checks and What They Do Not Show

Table 1 reports the current executable suite at the code snapshot used for this draft. A fictional demonstration DOCX produced 30 segmented records with 30 located source intervals. Local verification also passed 20 Python tests and eight JavaScript engine tests. These checks establish that the program runs on the listed fixtures and that the offset and review contracts are testable. They do not estimate precision, recall, error severity, or analyst benefit on real regulations. No held-out document corpus, inter-annotator agreement result, trained classifier, or comparative reviewer study is available.

The broader study should first pilot the annotation guide on diverse documents, then freeze a source-disjoint test split. Planned comparators are majority-class and ordered-rule baselines, a character n-gram linear model, a general encoder, and a legal-domain encoder such as LEGAL-BERT [5]. The literature cautions against assuming a larger model will dominate simple baselines on every legal extraction task [4]. Each comparator should use the same document split and output contract. Field-level span F1, change-impact recall, policy-outcome macro F1, calibration, and reviewer time require additional gold labels and prospective study designs; this paper reports none of them.

6 Limitations, Ethics, and Reproducibility

The baseline favors explicit English modal language. It may miss implicit duties, inherited actors, nested exceptions, tables, or obligations distributed across provisions. Clause-level evidence is narrower than field-level evidence: the current actor, action, and deadline strings are not individually span-validated. Greedy version matching and first-record policy comparison may fail on multi-clause documents. Approximate time conversion omits legal calendars and trigger semantics. A missing source interval is surfaced, but a present interval alone does not establish legal authority, correct interpretation, or compliance.

Regulatory text can guide consequential work. The interface therefore presents its outputs as review signals, preserves corrections, and avoids an automatic compliance certification claim. The local store is not designed for confidential multi-user deployment. Any future human study should obtain appropriate permissions, avoid collecting unnecessary personal data, and disclose whether reviewers saw system suggestions.

The implementation snapshot and synthetic suite are available through the public RegulaTrace repository and draft research branch at commit `9ae8ae8`.[1] The held-out evaluator and annotation rules are in `research/`. No annotated real-document corpus is released with this draft. The system uses exact source offsets within the supplied text but does not verify official source identity or PDF page coordinates.

7 Conclusion

RegulaTrace provides a transparent starting point for regulatory obligation analysis: a source-anchored record, explicit heuristic extraction and triage, and an auditable correction loop. The present evidence supports software reproducibility and synthetic-case behavior. Whether the system improves detection, interpretation, or reviewer work on real documents remains an open empirical question. The provided protocol makes that next claim falsifiable.

References

[1] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. Katz, and N. Aletras. LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of ACL*, pages 4310–4330, 2022. doi:10.18653/v1/2022.acl-long.297.

[2] Y. Koreeda and C. D. Manning. ContractNLI: A dataset for document-level natural language inference for contracts. In *Findings of EMNLP*, pages 1907–1919, 2021. doi:10.18653/v1/2021.findings-emnlp.164.

[3] K. Minkova, S. M. Chakravarthy, and G. van Dijck. Low-resource deontic modality classification in EU legislation. In *Proceedings of NLLP*, 2023. https://aclanthology.org/2023.nllp-1.15/.

[4] J. Piskorski and D. Skotarczak. Extraction of information provision activity requirements from EU acquis. In *Proceedings of EMNLP: Industry Track*, pages 1451–1471, 2025. doi:10.18653/v1/2025.emnlp-industry.102.

[5] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos. LEGAL-BERT: The Muppets straight out of law school. In *Findings of EMNLP*, pages 2898–2904, 2020. doi:10.18653/v1/2020.findings-emnlp.261.

[1] Source repository; implementation snapshot.